- Establish a dedicated meeting log for the project and record the following details for every meeting:
  - **Date**: The date of the meeting
  - **Time**: The start and end times of the meeting
  - **Location**: The physical or virtual location of the meeting
  - **Objective**: The main objective or agenda of the meeting
  - **Team members**: List of all team members present at the meeting
  - **Tasks allocated**: List specific tasks assigned to each team member, including deadlines
  - **Task completion confirmation:** Starting from the second meeting, confirm whether each team member completed the tasks allocated to them in previous meetings

Meeting 1 Notes

- Date: (9/30/2026)
- Time: 11:00-12:00
- Location: LEEP 2324
- Objective: Ensure each individual member fully understands the requirements of the assignment. Divide the roles/responsibilities to each member so that every aspect of the project is encompassed. Finish the initial set up for the repository, so all members can clone the repo and start off with the same starter code.
- Team members: everyone
- Tasks allocated:
    - Ellie: Implement the Easy AI mode for the project
    - Megan and Jana: Implement the Medium AI mode for the project
    - Serom: Integrates the AI system with the existing Game class and manages the overall game flow.
    - Sneha: Implement the hard AI mode for the project
    - Arpa: Implement a sound effects feature for the new feature implementation aspect of the project
    - Emma D: Write documentation and fix any features/implementations not done by the previous group
    - Emma R: Write documentation and create a UML diagram that encompasses both old and new features of the project.

Meeting 2 Notes

- Date: (10/06/2026)
- Time: 2:00-3:00
- Location: LEEP Atrium
- Objective: Project Status update - discuss blockers, update one another on progress, and continue working on individual tasks.
- Team members: everyone
- Tasks allocated:
    - Discussed certain files/dependencies that needed to be completed before another team member could work on/finish their implementations
    - Ensured that enough progress had been made to show the TA a demo at the next scrum meeting.
    - Talked about various blockers faced and how to overcome them.

Minesweeper file structure implementation:

```
minesweeper/

├── docs/
│   ├── Architecture.png            # Component diagrams, data flow, data structures
│   ├── CustomFeatureUML.png        # UML diagram for custom addition
│   ├── PersonHoursEstimate.md      # Estimated hours + methodology
│   └── ActualPersonHours.md        # Daily hours for all team members
│
├── src/
│   ├── __init__.py
│   ├── cell.py                     # Cell state model
│   ├── board.py                    # Existing board functionality + fixes
│   ├── mine_generator.py           # Existing mine generation + fixes
│   ├── number_calculator.py        # Existing number calculation + fixes
│   ├── game.py                     # Existing gameplay + AI integration
│   ├── ui.py                       # UI + AI difficulty selection
│   ├── input_handler.py            # Player input handling
│   ├── ai_solver.py                # Main AI controller
│   ├── easy_ai.py                  # Easy AI
│   ├── medium_ai.py                # Medium AI
│   ├── hard_ai.py                  # Hard AI / 1-2-1 rule
│   └── sound_effects.py            # Custom Addition
│
├── tests/
│   ├── test_board.py
│   ├── test_game.py
│   ├── test_easy_ai.py
│   ├── test_medium_ai.py
│   ├── test_hard_ai.py
│
├── README.md                       # Setup & run instructions
└── requirements.txt                # Dependencies (if any)
```

1. AI Controller & Easy AI - ELLIE

   AISolver(board, difficulty)

   getAvailableCells()

   makeMove()

   randomMove()

Creates the main structure used by all three AI difficulty levels. The AI controller determines which difficulty is currently selected and sends the AI turn to the appropriate solver. getAvailableCells() finds cells that are still covered and not flagged so the AI never attempts an illegal move.

Also implements the Easy AI. Easy mode uses randomMove() to randomly select one of the available covered and unflagged cells and uncover it. No logical reasoning is required for Easy mode.

2. Medium AI – Mine Detection Rule - Jana

   getHiddenNeighbors(row, col)

   countHiddenNeighbors(row, col)

   flagCertainMines()

   applyMineRule()

Implements the first logical rule required for Medium AI. The solver examines revealed numbered cells and determines how many neighboring cells are still hidden and how many neighboring mines have already been identified.

If the remaining hidden neighbors must all be mines in order to satisfy the revealed cell's number, the AI flags those cells.

This module communicates its decisions back to the main AI controller rather than controlling the entire game.

3. Medium AI – Safe Cell Detection Rule - Megan

countFlaggedNeighbors(row, col)

getSafeNeighbors(row, col)

openCertainSafeCells()

applySafeRule()

Implements the second logical rule required for Medium AI. The solver examines revealed numbered cells and compares their number with the number of neighboring cells already flagged as mines.

If the number of flagged neighbors equals the revealed cell's number, every other covered neighboring cell can safely be uncovered.

If neither Medium AI rule produces a move, control returns to the AI controller so it can perform a random valid move as a fallback.

4. Hard AI – 1-2-1 Pattern - Sneha

find121Pattern()

validate121Pattern()

flagOuterMines()

openInnerSafeCell()

makeHardMove()

Implements the additional logic required for Hard AI. Hard mode first attempts all of the Medium-level deduction rules before searching the board for valid 1-2-1 patterns.

When three side-by-side revealed cells form a valid 1-2-1 configuration, the AI analyzes the corresponding hidden cells. The two outer hidden neighbors are identified as mines and flagged, while the inner hidden neighbor is identified as safe and uncovered.

If neither the Medium rules nor the 1-2-1 rule can produce a move, Hard AI falls back to a random valid move.

5. Game Integration & AI Turns - Serom

startGame(mode, difficulty)

playerTurn()

aiTurn()

switchTurn()

getStatus()

restartGame()

Integrates the AI system with the existing Game class and manages the overall game flow.

In interactive mode, turns alternate between the human player and the AI. After the player's valid move is completed, switchTurn() transfers control to the AI. The AI performs its move before returning control to the player.

This portion also makes sure win/loss detection occurs after both player and AI moves and that the game stops accepting moves after a victory or loss.

This section also handles integration problems that occur when connecting the new AI modules with the inherited Project 1 code.

6. Custom Addition – Sound Effects -Arpa Das

playUncoverSound()

playFlagSound()

playMineSound()

playWinSound()

Adds Sound Effects as the custom Project 2 feature. Different sounds will play based on actions and events that happen during the game.

When a player uncovers a normal cell, playUncoverSound() plays a short sound effect. When a player places or removes a flag, playFlagSound() plays a different sound. If the player uncovers a mine, playMineSound() plays a game-over sound. When the player successfully clears the board, playWinSound() plays a victory sound.

The sound effects should respond to events from the Game controller without changing any of the actual Minesweeper game logic.

This section also includes creating the required UML diagram showing how the Player, UI, Game controller, and Sound Effect system interact when different game events occur.

7. UI, AI Selection & Input Handling

selectGameMode()

selectAIDifficulty()

InputHandler(event)

displayTurn()

displayDifficulty()

Updates the inherited user interface so the new Project 2 functionality is intuitive and usable without requiring a manual.

The player should be able to select the appropriate game mode and choose between Easy, Medium, and Hard AI difficulty. The UI displays whose turn it is during interactive gameplay and shows the currently selected AI difficulty.

InputHandler(event) interprets player input and returns the requested action. The main game loop/Game controller determines what happens with the request rather than putting game logic directly inside InputHandler.

8. Testing, Project 1 Bug Fixes & System Documentation - Emma R & Emma D

testProject1Functionality()

testEasyAI()

testMediumAI()

testHardAI()

stressTest()

Verifies that all inherited Project 1 functionality still works correctly after the Project 2 modifications. Any bugs discovered in the inherited system should be documented and fixed or assigned to the person responsible for the affected module.

Testing should cover normal gameplay, mine placement, first-click safety, recursive uncovering, flagging, win/loss detection, restarting, AI moves, and invalid inputs.

Stress testing should also be performed to identify crashes, memory issues, unexpected behavior, or problems caused by repeated actions.

This section also organizes the system documentation, including the updated architecture diagram, descriptions of system components, data flow, and important data structures.

The person-hours documents should be organized here, but each team member is responsible for keeping an accurate day-by-day record of their own hours.

EVERYONE'S RESPONSIBILITY

1. Document the files/functions they create or modify.

2. Add the required prologue comments to files they work on.

3. Include the function, class, or module name and a brief description.

4. Document inputs and outputs.

5. Attribute generative AI, StackOverflow, or any other external sources used.

6. Include their full name and creation date where required.

7. Comment major sections of their code so future teams can understand the logic.

8. Clearly identify whether code is original, sourced, or combined.

9. Record their own person-hours every day they work on the project.

10. Test their own feature before it is merged.

11. Help fix bugs related to the module or feature they worked on.

12. Make sure their code is committed before the final code freeze.

| Name | Task | Estimated hours | Actual hours | Date |
|---|---|---|---|---|
| Everyone | Understanding the Assignment | 1 | 0.5 | 9/30/26 |
| Everyone | Meet with TA and consolidate tasks | 1 | 1 | 9/30/26 |
| Ellie Thach | Implement ai_controller.py and easy_ai.py files | 2 | 2 | 10/4/26 |
| Megan Svoren | Implement medium_ai.py file | 3 | 2 | 10/5/26 |
| Jana Frady | Implement the first part of medium_ai.py | 2 | 2 | 10/6/26 |
| Serom Kim | Implement new functionalities and features to game.py and index.py | 2 | 2 | 10/6/26 |
| Emma Du | Complete team documentation | 2 | 2 | 10/10/26 |
| Emma Roy | Complete documentation of group meeting notes and uml diagram | 2 | 2 | 10/10/26 |
| Sneha Thomas | Implement hard ai with hardai.py, implemented interactive and auto modes | 3 | 3 | 10/6/26 |
| Arpa Das | Implemented sound effects | 2 | 1.5 | 10/5/26 |

Date: October 4, 2026
Time: 3pm
Location: Apartment
Objective: Implement the AI controller and Easy AI functionality, including selecting random valid moves and connecting the AI difficulty levels.
Team Members Present: Ellie Thach
Task Completion:

- Completed `ai_controller.py` and `easy_ai.py`.
- Implemented Easy AI to randomly select covered, unflagged cells.
- Established the AI controller structure for managing different AI difficulty levels.

---

Date: October 5, 2026
Time: 7pm
Location: Apartment
Objective: Implement Medium AI safe-cell detection rules, including revealing all hidden neighbors when the number of flagged neighbors equals the revealed cell's number.
Team Members Present: Megan Svoren
Task Completion:

- Implemented the safe-cell detection logic in `medium_ai.py`.
- Added functionality to identify and uncover neighboring cells determined to be safe.
- Incorporated logic for Medium AI to make deductions based on flagged neighboring mines.

---

Date: October 6, 2026
Time: 11am
Location: Library
Objective: Implement the first part of Medium AI, including identifying and flagging hidden neighboring cells that can logically be determined to contain mines.
Team Members Present: Jana Frady
Task Completion:

- Implemented the mine-detection portion of `medium_ai.py`.
- Added logic to evaluate hidden and flagged neighbors of revealed numbered cells.
- Implemented the rule for flagging hidden neighbors when all remaining hidden cells must contain mines.

---

Date: October 6, 2026
Time: 4pm
Location: House

Objective: Integrate new AI functionality into the existing Minesweeper game and update the game flow to support AI gameplay.
Team Members Present: Serom Kim
Task Completion:

- Implemented new functionality and features in `game.py` and `index.py`.
- Integrated AI gameplay into the existing game structure.
- Updated game flow to support interactive and AI-controlled gameplay.

---

Date: October 6, 2026
Time: 1pm
Location: Library
Objective: Implement Hard AI functionality using advanced Minesweeper deduction rules and support for interactive and automatic gameplay modes.
Team Members Present: Sneha Thomas
Task Completion:

- Implemented Hard AI functionality in `hardai.py`.
- Incorporated advanced deduction logic, including the 1-2-1 pattern rule.
- Implemented support for interactive and automatic AI gameplay modes.

---

Date: October 5, 2026
Time: 7pm
Location: House
Objective: Design and implement sound effects for revealing cells, flagging cells, uncovering mines, and winning the game.
Team Members Present: Arpa Das
Task Completion:

- Implemented sound effects for uncovering cells and placing or removing flags.
- Added distinct sound effects for encountering a mine and winning the game.
- Integrated sound effects into the Minesweeper gameplay experience.

---

Date: October 10, 2026
Time: 8pm
Location: House
Objective: Complete team documentation for Project 2, including individual contributions, task completion records, and person-hour tracking.
Team Members Present: Emma Du
Task Completion:

- Completed team documentation outlining individual responsibilities and contributions.
- Organized estimated and actual person-hours for each team member.

- Documented task completion details to support Project 2 submission requirements.

---

Date: October 10, 2026
Time: 4pm
Location: House
Objective: Complete documentation of group meetings and create an updated UML diagram reflecting the existing and newly implemented Minesweeper features.
Team Members Present: Emma Roy
Task Completion:

- Documented team meeting notes, including project objectives, task assignments, progress updates, and blockers.
- Created a UML diagram encompassing the original Minesweeper functionality and new Project 2 features.
- Organized meeting documentation to reflect team collaboration and development progress.